MACHINE LEARNING & ADVANCED MACHINE LEARNING

25 Practical Programs — Complete Python Code and Run Instructions

Practical range	Topics
1–4	Decision Tree
5	Cross-Validation (LOOCV)
6	Naive Bayes
7–9	Support Vector Machine (SVM)
10–11	Logistic and Linear Regression
12–16	K-Means and DBSCAN
17–21	Principal Component Analysis (PCA)
22–25	Linear Discriminant Analysis (LDA)

How to use this guide

Every practical has its own imports, complete code, and a function call at the end. Copy the code for the practical you need into a separate .py file, save it, and run it. The code is intentionally repeated so each practical can run independently.

Software setup

```
python --version
python -m pip install numpy pandas matplotlib scikit-learn
```

Use Python 3.9 or newer if possible. If your system uses the command **python3** instead of **python**, replace python with python3 in the run commands.

Run a practical

```
1. Open VS Code / IDLE / PyCharm.
2. Create a new Python file, e.g. practical_01.py.
3. Copy the complete code for that practical from this PDF.
4. Save the file.
5. Open the terminal in that file's folder.
6. Run: python practical_01.py
7. For graph-based practicals, a Matplotlib window will open. Close it to continue.
```

CSV files needed

Practical 1 and 4: **diabetes.csv** with target column **label**. Practical 7: **Social_Network_Ads.csv** with **Age**, **EstimatedSalary**, and **Purchased**. Practical 11: **Salary.csv** with **years_of_experience** and **salary**. Keep each CSV in the same folder as the Python file. The other practicals use datasets generated or loaded from scikit-learn.

Important exam note

The practical sheet specifies some dataset filenames and columns but does not provide the CSV files. You must use the dataset supplied by your institute and check its exact column names. For small synthetic datasets, accuracy values can vary if you change the random_state or split.

PRACTICAL 01

Decision Tree Classifier on diabetes.csv (maximum depth = 3)

Aim: Train a Decision Tree using every column except label, fit on the complete dataset, limit the depth to 3, and draw the tree.

Dataset requirement: Place diabetes.csv beside the Python file. It must contain the columns pregnant, glucose, bp, skin, insulin, bmi, pedigree, age, and label. The target column must be named label.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_01():
    df = pd.read_csv("diabetes.csv")
    X = df.drop("label", axis=1)
    y = df["label"]

    model = DecisionTreeClassifier(max_depth=3, random_state=42)
    model.fit(X, y)  # Complete dataset used for training

    plt.figure(figsize=(18, 9))
    plot_tree(model, feature_names=X.columns, class_names=["0", "1"],
              filled=True, rounded=True)
    plt.title("Decision Tree — Diabetes Dataset (max_depth=3)")
    plt.tight_layout()
    plt.show()

practical_01()
```

How to run: Save this practical's code as `practical_01.py`, then run `python practical_01.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 02

Compare Decision Tree and SVM on the Iris dataset

Aim: Train Decision Tree and SVM classifiers on Iris and print their test accuracies.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_02():
    iris = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(
        iris.data, iris.target, test_size=0.2, random_state=42,
        stratify=iris.target
    )

    dt = DecisionTreeClassifier(random_state=42)
    svm = SVC(kernel="rbf")

    dt.fit(X_train, y_train)
    svm.fit(X_train, y_train)

    print("Decision Tree Accuracy:", accuracy_score(y_test, dt.predict(X_test)))
    print("SVM Accuracy:", accuracy_score(y_test, svm.predict(X_test)))

practical_02()
```

How to run: Save this practical's code as `practical_02.py`, then run `python practical_02.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 03

Compare two Decision Trees on Iris (max leaf nodes vs minimum samples per leaf)

Aim: Build two Iris Decision Trees with max_leaf_nodes=8 and min_samples_leaf=5, then compare their accuracies.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_03():
    iris = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(
        iris.data, iris.target, test_size=0.2, random_state=42,
        stratify=iris.target
    )

    tree_max_leaf = DecisionTreeClassifier(max_leaf_nodes=8, random_state=42)
    tree_min_sample = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)

    tree_max_leaf.fit(X_train, y_train)
    tree_min_sample.fit(X_train, y_train)

    print("Accuracy (max_leaf_nodes=8):",
          accuracy_score(y_test, tree_max_leaf.predict(X_test)))
    print("Accuracy (min_samples_leaf=5):",
          accuracy_score(y_test, tree_min_sample.predict(X_test)))

practical_03()
```

How to run: Save this practical's code as `practical_03.py`, then run `python practical_03.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 04

Compare Decision Tree and Logistic Regression on diabetes.csv

Aim: Train Decision Tree and Logistic Regression models on diabetes.csv and compare test accuracy.

Dataset requirement: Place diabetes.csv beside the Python file and ensure the target column is named label. The code uses an 80/20 train-test split.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_04():
    df = pd.read_csv("diabetes.csv")
    X = df.drop("label", axis=1)
    y = df["label"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    dt = DecisionTreeClassifier(random_state=42)
    lr = LogisticRegression(max_iter=2000)

    dt.fit(X_train, y_train)
    lr.fit(X_train, y_train)

    print("Decision Tree Accuracy:", accuracy_score(y_test, dt.predict(X_test)))
    print("Logistic Regression Accuracy:", accuracy_score(y_test, lr.predict(X_test)))

practical_04()
```

How to run: Save this practical's code as `practical_04.py`, then run `python practical_04.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 05

Leave-One-Out Cross-Validation (LOOCV) using a Decision Tree

Aim: Create 100 samples with 5 features and a binary target. Evaluate a Decision Tree using Leave-One-Out Cross-Validation and print every iteration's result and the mean accuracy.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_05():
    X, y = make_classification(
        n_samples=100, n_features=5, n_informative=3,
        n_redundant=0, n_classes=2, random_state=42
    )

    loo = LeaveOneOut()
    model = DecisionTreeClassifier(random_state=42)
    scores = []

    for iteration, (train_index, test_index) in enumerate(loo.split(X), start=1):
        X_train, X_test = X[train_index], X[test_index]
        y_train, y_test = y[train_index], y[test_index]

        model.fit(X_train, y_train)
        prediction = model.predict(X_test)
        score = accuracy_score(y_test, prediction)
        scores.append(score)
        print(f"Iteration {iteration}: Accuracy = {score:.0f}")

    print("Average LOOCV Accuracy:", np.mean(scores))

practical_05()
```

How to run: Save this practical's code as `practical_05.py`, then run `python practical_05.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 06

Gaussian Naive Bayes on the Breast Cancer dataset

Aim: Load the sklearn breast cancer dataset, split it into training and testing sets, train Gaussian Naive Bayes, and predict test labels.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_06():
    data = load_breast_cancer()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.2, random_state=42,
        stratify=data.target
    )

    model = GaussianNB()
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print("Predicted labels:", predictions)
    print("Actual labels:   ", y_test)
    print("Accuracy:", accuracy_score(y_test, predictions))

practical_06()
```

How to run: Save this practical's code as `practical_06.py`, then run `python practical_06.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 07

Linear and RBF Support Vector Classifiers on Social_Network_Ads.csv

Aim: Use Age and EstimatedSalary to predict Purchased. Train linear and RBF SVC models using a 30% test split and compare accuracy.

Dataset requirement: Place Social_Network_Ads.csv beside the Python file. It must contain Age, EstimatedSalary, and Purchased. UserID and Gender are not used.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_07():
    df = pd.read_csv("Social_Network_Ads.csv")
    X = df[["Age", "EstimatedSalary"]]
    y = df["Purchased"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.30, random_state=42, stratify=y
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    linear_svc = SVC(kernel="linear", C=1.0)
    rbf_svc = SVC(kernel="rbf", C=1.0, gamma="scale")

    for name, model in [("Linear SVC", linear_svc), ("RBF SVC", rbf_svc)]:
        model.fit(X_train_scaled, y_train)
        prediction = model.predict(X_test_scaled)
        print(name, "Accuracy:", accuracy_score(y_test, prediction))

practical_07()
```

How to run: Save this practical's code as `practical_07.py`, then run `python practical_07.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 08

SVC on the Breast Cancer dataset with result visualization

Aim: Train an SVC on all 30 breast cancer features and visualize classification results with a confusion matrix.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_08():
    data = load_breast_cancer()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.2, random_state=42,
        stratify=data.target
    )

    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    model = SVC(kernel="rbf", C=1.0, gamma="scale")
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print("SVC Accuracy:", accuracy_score(y_test, predictions))
    ConfusionMatrixDisplay.from_predictions(
        y_test, predictions, display_labels=data.target_names,
        cmap="Blues"
    )
    plt.title("SVC — Breast Cancer Confusion Matrix")
    plt.tight_layout()
    plt.show()

practical_08()
```

How to run: Save this practical's code as `practical_08.py`, then run `python practical_08.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 09

SVC on the Iris dataset with result visualization

Aim: Train an SVC to classify the three Iris species and visualize the test results in a 2D PCA view.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_09():
    data = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.2, random_state=42,
        stratify=data.target
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    model = SVC(kernel="rbf", C=1.0, gamma="scale")
    model.fit(X_train_scaled, y_train)
    predictions = model.predict(X_test_scaled)

    print("SVC Accuracy:", accuracy_score(y_test, predictions))

    # Reduce test data to 2D only for a simple result visualization.
    pca = PCA(n_components=2)
    X_test_2d = pca.fit_transform(X_test_scaled)
    plt.figure(figsize=(8, 6))
    for class_id, class_name in enumerate(data.target_names):
        mask = y_test == class_id
        plt.scatter(X_test_2d[mask, 0], X_test_2d[mask, 1],
                    label=f"Actual {class_name}", marker="o")
        pred_mask = predictions == class_id
        plt.scatter(X_test_2d[pred_mask, 0], X_test_2d[pred_mask, 1],
                    label=f"Predicted {class_name}", marker="x")
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.title("Iris SVC — Actual and Predicted Test Classes (PCA view)")
    plt.legend()
    plt.tight_layout()
    plt.show()

practical_09()
```

How to run: Save this practical's code as `practical_09.py`, then run `python practical_09.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 10

Logistic Regression for customer churn prediction

Aim: Use the customer data given in the practical sheet to train Logistic Regression with 80% training data, predict churn, and print accuracy.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_10():
    data = {
        "Age": [25, 34, 45, 29, 50, 38, 42, 35, 48, 55],
        "Account_Length": [12, 24, 36, 18, 48, 30, 42, 24, 36, 60],
        "Monthly_Charges": [70, 90, 80, 100, 60, 80, 90, 70, 80, 100],
        "Churn": [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
    }
    df = pd.DataFrame(data)
    X = df[["Age", "Account_Length", "Monthly_Charges"]]
    y = df["Churn"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    model = LogisticRegression(max_iter=2000)
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)
    probabilities = model.predict_proba(X_test)[:, 1]

    print("Predicted churn labels:", predictions)
    print("Churn probabilities:", probabilities)
    print("Accuracy:", accuracy_score(y_test, predictions))

practical_10()
```

How to run: Save this practical's code as `practical_10.py`, then run `python practical_10.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 11

Simple Linear Regression on Salary.csv with residual plot

Aim: Fit simple Linear Regression to all rows in Salary.csv. Plot the fitted line with data points and a residual plot.

Dataset requirement: Place Salary.csv beside the Python file. It must contain years_of_experience and salary columns.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_11():
    df = pd.read_csv("Salary.csv")
    X = df[["years_of_experience"]]
    y = df["salary"]

    model = LinearRegression()
    model.fit(X, y)
    predictions = model.predict(X)
    residuals = y - predictions

    print("Slope:", model.coef_[0])
    print("Intercept:", model.intercept_)

    plt.figure(figsize=(8, 5))
    plt.scatter(X, y, label="Actual data")
    plt.plot(X, predictions, label="Best-fit line")
    plt.xlabel("Years of Experience")
    plt.ylabel("Salary")
    plt.title("Simple Linear Regression – Salary")
    plt.legend()
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(8, 5))
    plt.scatter(predictions, residuals)
    plt.axhline(y=0, linestyle="--")
    plt.xlabel("Predicted Salary")
    plt.ylabel("Residuals")
    plt.title("Residual Plot")
    plt.tight_layout()
    plt.show()

practical_11()
```

How to run: Save this practical's code as `practical_11.py`, then run `python practical_11.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 12

Elbow Method to find the optimal number of clusters (14 points)

Aim: Generate 14 two-dimensional points, calculate K-Means inertia for several K values, and plot the Elbow curve.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_12():
    X = np.array([
        [1, 2], [2, 1], [2, 3], [3, 2],
        [8, 8], [9, 8], [8, 9], [9, 10],
        [15, 2], [16, 3], [15, 4], [17, 2],
        [5, 14], [6, 15]
    ])

    wcss = []
    k_values = range(1, 8)
    for k in k_values:
        model = KMeans(n_clusters=k, random_state=42, n_init=10)
        model.fit(X)
        wcss.append(model.inertia_)

    plt.plot(list(k_values), wcss, marker="o")
    plt.xlabel("Number of Clusters (K)")
    plt.ylabel("WCSS / Inertia")
    plt.title("Elbow Method — 14 Data Points")
    plt.xticks(list(k_values))
    plt.tight_layout()
    plt.show()

practical_12()
```

How to run: Save this practical's code as `practical_12.py`, then run `python practical_12.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 13

K-Means on 300 generated samples for K = 2 to 7

Aim: Generate 300 samples with make_blobs and show K-Means results for K values 2 through 7.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_13():
    X, _ = make_blobs(n_samples=300, centers=4, cluster_std=1.0,
                      random_state=42)

    fig, axes = plt.subplots(2, 3, figsize=(15, 9))
    for ax, k in zip(axes.ravel(), range(2, 8)):
        model = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = model.fit_predict(X)
        ax.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis", s=25)
        ax.scatter(model.cluster_centers_[:, 0], model.cluster_centers_[:, 1],
                   marker="X", s=180, edgecolor="black")
        ax.set_title(f"KMeans with K={k}")
        ax.set_xlabel("Feature 1")
        ax.set_ylabel("Feature 2")

    plt.tight_layout()
    plt.show()

practical_13()
```

How to run: Save this practical's code as `practical_13.py`, then run `python practical_13.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 14

K-Means on a random dataset of 35 points

Aim: Generate 35 random points and visualize how K-Means clustering changes for K values 2 through 7.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_14():
    rng = np.random.default_rng(42)
    X = rng.uniform(0, 20, size=(35, 2))

    fig, axes = plt.subplots(2, 3, figsize=(15, 9))
    for ax, k in zip(axes.ravel(), range(2, 8)):
        model = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = model.fit_predict(X)
        ax.scatter(X[:, 0], X[:, 1], c=labels, cmap="tab10", s=55)
        ax.scatter(model.cluster_centers_[:, 0], model.cluster_centers_[:, 1],
                   marker="X", s=160, edgecolor="black")
        ax.set_title(f"Random 35 points — K={k}")
        ax.set_xlabel("X")
        ax.set_ylabel("Y")

    plt.tight_layout()
    plt.show()
    print("Impact: increasing K creates more, smaller clusters and reduces "
          "within-cluster variation, but may over-segment the data.")

practical_14()
```

How to run: Save this practical's code as `practical_14.py`, then run `python practical_14.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 15

DBSCAN clustering using the make_moons dataset

Aim: Apply DBSCAN to a two-dimensional make_moons dataset and plot the clusters. Label -1 represents noise.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_15():
    X, _ = make_moons(n_samples=300, noise=0.08, random_state=42)

    model = DBSCAN(eps=0.2, min_samples=5)
    labels = model.fit_predict(X)

    plt.figure(figsize=(8, 6))
    plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis", s=30)
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.title("DBSCAN Clustering — make_moons")
    plt.tight_layout()
    plt.show()

    print("Cluster labels (-1 means noise):", np.unique(labels))

practical_15()
```

How to run: Save this practical's code as `practical_15.py`, then run `python practical_15.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 16

DBSCAN on columns 3 and 4 of a five-column random dataset

Aim: Generate five columns of random data, select columns 3 and 4, and apply DBSCAN with eps=5 and min_samples=5.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_16():
    rng = np.random.default_rng(42)
    data = rng.uniform(0, 20, size=(100, 5))
    df = pd.DataFrame(data, columns=["Column1", "Column2", "Column3",
                                     "Column4", "Column5"])
    X = df[["Column3", "Column4"]].values

    model = DBSCAN(eps=5, min_samples=5)
    labels = model.fit_predict(X)

    plt.figure(figsize=(8, 6))
    plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis", s=35)
    plt.xlabel("Column 3")
    plt.ylabel("Column 4")
    plt.title("DBSCAN on Random Dataset (eps=5, min_samples=5)")
    plt.tight_layout()
    plt.show()

    print("Unique labels (-1 indicates noise):", np.unique(labels))

practical_16()
```

How to run: Save this practical's code as `practical_16.py`, then run `python practical_16.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 17

PCA on standardized Iris data (4 dimensions to 2)

Aim: Standardize the Iris features, reduce them to two principal components, and plot the three classes.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_17():
    data = load_iris()
    X = StandardScaler().fit_transform(data.data)
    y = data.target

    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X)

    plt.figure(figsize=(8, 6))
    for i, name in enumerate(data.target_names):
        plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1], label=name)
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.title("PCA on Standardized Iris Dataset")
    plt.legend()
    plt.tight_layout()
    plt.show()
    print("Explained variance ratio:", pca.explained_variance_ratio_)

practical_17()
```

How to run: Save this practical's code as `practical_17.py`, then run `python practical_17.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 18

Explore Iris and visualize PCA-reduced data in 2D

Aim: Print the Iris dataset shape and first rows, then visualize the two-dimensional PCA transformation by species.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_18():
    data = load_iris()
    df = pd.DataFrame(data.data, columns=data.feature_names)
    df["target"] = data.target

    print("Dataset shape:", df.shape)
    print(df.head())

    # Standardization is recommended before PCA because features have
    # different scales.
    X = StandardScaler().fit_transform(data.data)
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X)

    plt.figure(figsize=(8, 6))
    for i, name in enumerate(data.target_names):
        plt.scatter(X_pca[data.target == i, 0],
                    X_pca[data.target == i, 1], label=name)
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.title("Iris Dataset — PCA 2D")
    plt.legend()
    plt.tight_layout()
    plt.show()

practical_18()
```

How to run: Save this practical's code as `practical_18.py`, then run `python practical_18.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 19

PCA on 100 random samples with 3 features

Aim: Generate 100 rows with 3 random features, apply PCA to reduce to 2 dimensions, and display the transformed data.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_19():
    rng = np.random.default_rng(42)
    X = rng.random((100, 3))

    pca = PCA(n_components=2)
    X_reduced = pca.fit_transform(X)

    print("Original shape:", X.shape)
    print("Reduced shape:", X_reduced.shape)
    print("First five reduced rows:\n", X_reduced[:5])

    plt.figure(figsize=(8, 6))
    plt.scatter(X_reduced[:, 0], X_reduced[:, 1])
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.title("PCA on 100 Random Samples (3 Features to 2)")
    plt.tight_layout()
    plt.show()

practical_19()
```

How to run: Save this practical's code as `practical_19.py`, then run `python practical_19.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 20

PCA on a selected dataset and explained variance

Aim: Apply PCA to the Wine dataset and print the explained variance ratio of every principal component.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_20():
    data = load_wine()
    X = StandardScaler().fit_transform(data.data)

    pca = PCA()
    X_pca = pca.fit_transform(X)

    print("Transformed shape:", X_pca.shape)
    print("Explained variance ratio of all components:")
    print(pca.explained_variance_ratio_)
    print("Total explained variance:", pca.explained_variance_ratio_.sum())

practical_20()
```

How to run: Save this practical's code as `practical_20.py`, then run `python practical_20.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 21

PCA on Iris using two principal components

Aim: Load Iris, initialize PCA with n_components=2, and plot the reduced dataset.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_21():
    iris = load_iris()
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(iris.data)

    plt.figure(figsize=(8, 6))
    scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1],
                          c=iris.target, cmap="viridis")
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.title("PCA — Iris Dataset (2 Components)")
    plt.legend(handles=scatter.legend_elements()[0],
               labels=list(iris.target_names), title="Species")
    plt.tight_layout()
    plt.show()

practical_21()
```

How to run: Save this practical's code as `practical_21.py`, then run `python practical_21.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 22

LDA on the Wine dataset (2 components)

Aim: Load Wine, apply LDA with two components, and plot the reduced data with one color per class.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_22():
    data = load_wine()
    lda = LinearDiscriminantAnalysis(n_components=2)
    X_lda = lda.fit_transform(data.data, data.target)

    plt.figure(figsize=(8, 6))
    for i, name in enumerate(data.target_names):
        plt.scatter(X_lda[data.target == i, 0],
                    X_lda[data.target == i, 1], label=name)
    plt.xlabel("Linear Discriminant 1")
    plt.ylabel("Linear Discriminant 2")
    plt.title("LDA — Wine Dataset (2 Components)")
    plt.legend()
    plt.tight_layout()
    plt.show()

practical_22()
```

How to run: Save this practical's code as `practical_22.py`, then run `python practical_22.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 23

LDA on the Breast Cancer dataset (1 component)

Aim: Load Breast Cancer, apply LDA with one component, and plot the one-dimensional representation.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_23():
    data = load_breast_cancer()
    lda = LinearDiscriminantAnalysis(n_components=1)
    X_lda = lda.fit_transform(data.data, data.target)

    plt.figure(figsize=(8, 5))
    for i, name in enumerate(data.target_names):
        plt.scatter(X_lda[data.target == i, 0],
                    np.zeros(np.sum(data.target == i)) + i,
                    label=name, alpha=0.7)
    plt.xlabel("Linear Discriminant 1")
    plt.yticks([0, 1], list(data.target_names))
    plt.title("LDA — Breast Cancer Dataset (1 Component)")
    plt.legend()
    plt.tight_layout()
    plt.show()

practical_23()
```

How to run: Save this practical's code as `practical_23.py`, then run `python practical_23.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 24

LDA on the Digits dataset (2 components)

Aim: Load Digits, apply LDA with two components, and plot the reduced dataset.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_24():
    data = load_digits()
    lda = LinearDiscriminantAnalysis(n_components=2)
    X_lda = lda.fit_transform(data.data, data.target)

    plt.figure(figsize=(9, 7))
    scatter = plt.scatter(X_lda[:, 0], X_lda[:, 1],
                          c=data.target, cmap="tab10", s=15)
    plt.xlabel("Linear Discriminant 1")
    plt.ylabel("Linear Discriminant 2")
    plt.title("LDA — Digits Dataset (2 Components)")
    plt.colorbar(scatter, label="Digit class")
    plt.tight_layout()
    plt.show()

practical_24()
```

How to run: Save this practical's code as `practical_24.py`, then run `python practical_24.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.

PRACTICAL 25

LDA on the Iris dataset (2 components)

Aim: Load Iris, apply LDA with two components, and plot the reduced dataset by species.

Dataset: This program uses a built-in scikit-learn dataset or generates data as described in the practical.

Complete standalone Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import (
    load_iris, load_breast_cancer, load_wine, load_digits,
    make_classification, make_blobs, make_moons
)
from sklearn.model_selection import train_test_split, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


def practical_25():
    data = load_iris()
    lda = LinearDiscriminantAnalysis(n_components=2)
    X_lda = lda.fit_transform(data.data, data.target)

    plt.figure(figsize=(8, 6))
    for i, name in enumerate(data.target_names):
        plt.scatter(X_lda[data.target == i, 0],
                    X_lda[data.target == i, 1], label=name)
    plt.xlabel("Linear Discriminant 1")
    plt.ylabel("Linear Discriminant 2")
    plt.title("LDA — Iris Dataset (2 Components)")
    plt.legend()
    plt.tight_layout()
    plt.show()

practical_25()
```

How to run: Save this practical's code as `practical_25.py`, then run `python practical_25.py` in the terminal. Install the libraries once using the setup command on the first pages. If the program displays a graph, a plot window will appear.